

MSc Computer Science — Distributed Computing

Lab 02

Chord Distributed Hash Table

Go Language | 5-Node LAN Ring | 2 Hours

Your Group Details

Group Number			Role
_____			Node 1=creator 2-5=joiner

Learning Objectives

By the end of this lab you will be able to:

- Explain how consistent hashing distributes keys across a ring
- Implement the Chord lookup algorithm using finger tables
- Build a peer-to-peer ring that survives node joins and failures
- Observe $O(\log N)$ routing across a real 5-node LAN cluster
- Explain the role of `stabilize()` and why eventual consistency matters



Before This Lab

You should have completed Lab 01 (TCP client-server in Go) and understand goroutines, channels, and `net/rpc`. Review the Go Quick Reference on the last page if needed.

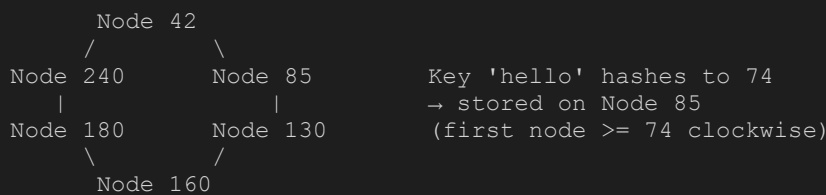
Background — What is Chord DHT?

A Chord DHT is a distributed key-value store where data is spread across multiple nodes automatically. Unlike Lab 01 (one central server), there is no single point of control — every node is equal.

The Ring

All nodes are arranged on a virtual ring of positions 0–255. Each node gets an ID by hashing its IP:port address. Keys are also hashed to a ring position, and stored on the nearest node clockwise.

Ring positions 0–255 ($2^8 = 256$ slots):



Finger Tables — $O(\log N)$ Lookup

Each node keeps a finger table of M shortcuts across the ring. Instead of asking every node one-by-one, a lookup jumps halfway across the ring each hop — reaching the right node in $O(\log N)$ hops.

Node at ID=42, ring size=256, $M=8$ finger entries:

```
finger[0]  start=43  → successor of 43  (1 step ahead)
finger[1]  start=44  → successor of 44  (2 steps ahead)
finger[2]  start=46  → successor of 46  (4 steps ahead)
finger[3]  start=50  → successor of 50  (8 steps ahead)
finger[4]  start=58  → successor of 58  (16 steps ahead)
finger[5]  start=74  → successor of 74  (32 steps ahead)
finger[6]  start=106 → successor of 106 (64 steps ahead)
finger[7]  start=170 → successor of 170 (128 steps ahead)
```

With 5 nodes, any key is found in at most 3 hops ($\log_2(5) \approx 2.3$)

Maintenance — How the Ring Fixes Itself

After a node joins or leaves, three background goroutines keep the ring correct:

Function	Runs every	What it does
stabilize()	500ms	Checks if a better successor exists, notifies successor
fixFingers()	500ms	Refreshes one finger table entry (rotates through all M)
checkPredecessor()	1s	Pings predecessor — clears it if it has failed

Your Chord Ring

Each group runs one 5-node Chord ring across all 5 lab machines. All 5 nodes communicate over real LAN using Go RPC.

Node	Machine IP	Role	Your container
Node 1	10.8.100.22	Creates the ring	lab02-gX-node1
Node 2	10.8.100.1	Joins via Node 1	lab02-gX-node2
Node 3	10.8.100.2	Joins via Node 1	lab02-gX-node3
Node 4	10.8.100.12	Joins via Node 1	lab02-gX-node4
Node 5	10.8.100.28	Joins via Node 1	lab02-gX-node5

Replace X with your group number. Example for Group 3: lab02-g3-node1

Getting Connected

Step 1 — SSH Into Your Assigned Machine

```
# Replace X with your group number
ssh groupX@10.8.100.22 # Node 1
ssh groupX@10.8.100.1  # Node 2
ssh groupX@10.8.100.2  # Node 3
ssh groupX@10.8.100.12 # Node 4
ssh groupX@10.8.100.28 # Node 5

# Password: groupX@123 (e.g. group3@123 for Group 3)
```

Step 2 — Enter Your Container

```
docker exec -it lab02-gX-node1 bash # Node 1
docker exec -it lab02-gX-node2 bash # Node 2
docker exec -it lab02-gX-node3 bash # Node 3
docker exec -it lab02-gX-node4 bash # Node 4
docker exec -it lab02-gX-node5 bash # Node 5

# You should see:
root@hostname:/lab02/chord#
```

Step 3 — Verify Node is Running

```
# The Chord node starts automatically — verify it is running
ps aux | grep chord_bin
# Should show: ./chord_bin -mode server -port 7000 -host 10.8.100.22

# Test with CLI (ping)
./chord_bin -mode cli -port 7000 ping
# Should show: Node is alive
```

 Important

The node server starts automatically when the container launches — do NOT run `./chord_bin -mode server` manually until you have killed the existing one with `pkill chord_bin`.

Step 4 — Test a Put and Get

```
# Replace 7000 with your group's port
./chord_bin -mode cli -port 7002 put mykey myvalue
# Should show: Stored key="mykey" value="myvalue"

./chord_bin -mode cli -port 7002 get mykey
# Should show: key="mykey" value="myvalue"      (after completing TODOs)

# Before completing TODOs — both commands return without error but do nothing
# This is expected and correct
```

Coding Tasks

All tasks are marked with TODO comments in the source files. Open and edit with nano:

```
nano /lab02/chord/node.go      # Tasks 1-2
nano /lab02/chord/ring.go     # Tasks 3-7
nano /lab02/chord/rpc.go      # Task 8
nano /lab02/chord/storage.go  # Tasks 9-11
```

Part 1 — Core Ring (Tasks 1–4)

Start here — these are the foundation everything else depends on.

Task

1

node.g
o

hashKey(key string) uint8 [Easy]

Hash any string to a ring position 0–255 using SHA-1

Task

2

node.g
o

inRange(id, start, end uint8) bool [Easy]

Check if id falls in the arc (start, end] — handle ring wraparound

Task

3

ring.g
o

findSuccessor(id uint8) [Medium]

Find which node owns a given ID — check local successor or forward to best finger

Task

4

ring.g
o

closestPrecedingFinger(id uint8) [Medium]

Walk finger table backwards to find the best node to forward a lookup to

Part 2 — Join and Maintenance (Tasks 5–7)

Once the ring lookup works, implement how nodes join and keep the ring healthy.

Task

5

ring.g
o

join(existingAddr string) [Medium]

Join the ring via a known node (or create a new ring if address is empty)

Task

6

ring.g
o

stabilize() [Hard]

Fix successor pointers — ask successor for its predecessor and update if needed

Task

7

ring.g
o

fixFingers() [Medium]

Refresh one finger table entry per call — rotate through all M fingers

Part 3 — RPC Handlers (Task 8)

Connect the ring logic to the network. Each handler is called remotely by other nodes.

Task 8
rpc.go

6 handlers [Easy]

FindSuccessor, GetPredecessor, Notify, Put, Get, Delete — each calls the correct local function

Part 4 — Distributed Storage (Tasks 9–11)

Route key-value operations to the correct node in the ring.

Task 9
storage.go

Put(key, value string) [Medium]

Hash the key, find its node, store locally or forward via RPC

Task 10
storage.go

Get(key string) [Medium]

Hash the key, find its node, retrieve locally or forward via RPC

Task 11
storage.go

Delete(key string) [Easy]

Hash the key, find its node, delete locally or forward via RPC

How to Recompile After Editing

⚠ Kill the Running Node First

Always kill the running server before recompiling — otherwise the new binary cannot bind to the port.

```
# Step 1 – kill running node
pkill chord_bin

# Step 2 – recompile
cd /lab02/chord
go build -o chord_bin .

# Step 3 – restart server (use YOUR group's port and host IP)
# Node 1:
./chord_bin -mode server -port 7002 -host 10.8.100.22 &

# Nodes 2-5 (must join via Node 1):
./chord_bin -mode server -port 7002 -host 10.8.100.1 -join 10.8.100.22:7002 &

# Step 4 – use CLI to test
./chord_bin -mode cli -port 7002 put hello world
./chord_bin -mode cli -port 7002 get hello
```

Expected Output After All Tasks Complete

```
# Node 1 server log shows:
[CHORD] Node ID = 42
[RPC] Server listening on port 7000
[CHORD] New ring created
[RING] Background maintenance started
[STORAGE] Stored key="hello" value="world" (hash=186 local)

# CLI output:
./chord_bin -mode cli -port 7002 put hello world
Stored key="hello" value="world"

./chord_bin -mode cli -port 7002 get hello
key="hello" value="world"

# Key stored on a DIFFERENT node (routed automatically):
./chord_bin -mode cli -port 7001 get hello (from Node 2)
key="hello" value="world" <- routed to correct node transparently
```

Experiments

Complete all 11 coding tasks first. Then run each experiment in order. Record results in the table at the end.

Experiment A — Baseline — Ring Working

Steps:

1. Make sure all 5 nodes are running and have joined the ring
2. Wait 5 seconds for stabilize() to run a few cycles
3. From Node 1: put 10 different key-value pairs
4. From Node 3: get all 10 keys back
5. Use the server logs to see which node each key was stored on

Observe and record:

- > Did all 10 gets succeed? Were any keys missing?
- > How many different nodes stored at least one key?
- > Did keys stored from Node 1 get retrieved correctly from Node 3?

Experiment B — Key Distribution — Consistent Hashing

Steps:

6. From any node, put 100 keys: key0 through key99
7. On each of the 5 nodes, check the server log output to count stored keys
8. Record how many keys landed on each node

Observe and record:

- > How evenly were the 100 keys distributed across 5 nodes?
- > Which node stored the most? The fewest?
- > Run 'hashKey' mentally on key0 — what position does it hash to?

Experiment C — Node Failure — What Breaks?

Steps:

9. Put 20 keys from Node 1 and confirm all can be retrieved
10. Kill Node 3: pkill chord_bin (run this inside Node 3 container)
11. Wait 5 seconds for checkPredecessor() to detect the failure
12. Try to retrieve all 20 keys again from Node 1
13. Try putting 5 new keys

Observe and record:

- > Which keys became unreachable after Node 3 failed?
- > Why those specific keys and not others?
- > Were new puts affected? What error appeared?
- > What would you need to add to survive node failure?

Experiment D — Node Join — Adding a 6th Node

Steps:

14. Restore your ring: ask your instructor to run restart_group.sh

15. Wait for ring to stabilize fully (30 seconds)
16. Re-add 20 keys from Node 1
17. Launch a 6th node on Machine 1 (different port):
18. `docker run -it --network host -e GROUP_ID=X -e NODE_NUM=6 -e NODE_PORT=<port+20> -e HOST_IP=10.8.100.22 -e JOIN_ADDR=10.8.100.22:<your_port> lab02 bash`
19. From inside: `./chord_bin -mode server -port <port+20> -host 10.8.100.22 -join 10.8.100.22:<your_port> &`
20. Wait 10 seconds and retrieve all 20 keys from the 6th node

Observe and record:

- > Did keys migrate to the 6th node after it joined?
- > How did the ring IDs change after the new node joined?
- > Did any key retrievals fail during the join process?

Experiment E — Routing Hops — $O(\log N)$ in Action**Steps:**

21. Put a key on Node 1 and note which node stores it (check server log)
22. Try to GET that key from every other node (2, 3, 4, 5)
23. Add some `log.Printf` statements to `findSuccessor()` to count hops
24. Rebuild and repeat — count how many RPC calls each lookup takes

Observe and record:

- > With 5 nodes, what is the maximum number of hops to find any key?
- > Did any lookup take more than $\log_2(5) \approx 3$ hops?
- > What would happen to hop count if you had 32 nodes instead of 5?

Discussion Questions

Answer individually after completing the experiments.

25. Why does consistent hashing minimise key movement when a node joins compared to simple modulo hashing ($\text{key} \% N$)? Give a concrete example with numbers.

Answer:

26. In Experiment C, when Node 3 failed, only some keys became unreachable. Explain exactly which keys those were and why — relate your answer to ring positions and node IDs.

Answer:

27. What is the role of `stabilize()`? What would happen to the ring if you commented out the `stabilize()` goroutine and then a new node joined?

Answer:

28. This implementation has no data replication — each key is stored on exactly one node. Describe how you would add replication to make the ring survive node failures.

Answer:

Results Recording

Experiment	Result / Observation	Notes
A — Baseline: All 10 gets succeeded?		
A — How many nodes stored keys?		
B — Key distribution (keys per node)	N1: N2: N3: N4: N5:	
C — Which keys became unreachable?		
C — Error message seen		
D — Did keys migrate to 6th node?		
E — Max hops for 5-node ring		
E — Expected hops ($\log_2 5 = ?$)		

Go Quick Reference — Chord DHT

SHA-1 Hashing

```
import "crypto/sha1"

// Hash a string to a uint8 (0-255)
func hashKey(key string) uint8 {
    h := sha1.Sum([]byte(key)) // returns [20]byte
    return h[0]                // first byte = ring position
}
```

Ring Arithmetic

```
// Check if id is in half-open arc (start, end] on ring
func inRange(id, start, end uint8) bool {
    if start == end { return true } // full ring
    if start < end { return id > start && id <= end }
    return id > start || id <= end // wraparound
}

// Finger table start position for index i:
start := n.info.ID + (uint8(1) << uint(i))
// Wrapping is automatic - uint8 wraps at 256
```

Go RPC

```
// Make an RPC call to another node
err := callRPC("10.8.100.1:7000", "ChordRPC.FindSuccessor",
    &FindSuccessorArgs{ID: 42},
    &FindSuccessorReply{})

// RPC handler - must match this exact signature
func (r *ChordRPC) FindSuccessor(args *FindSuccessorArgs, reply
    *FindSuccessorReply) error {
    node, err := r.node.findSuccessor(args.ID)
    if err != nil { return err }
    reply.Node = node
    return nil
}
```

Mutex Patterns

```
// Read lock - multiple goroutines can read simultaneously
n.mu.RLock()
succ := n.fingers[0] // safe to read
n.mu.RUnlock()

// Write lock - exclusive access for writes
n.mu.Lock()
n.fingers[0] = newSuccessor // safe to write
n.mu.Unlock()

// IMPORTANT: never call RPC while holding a lock
// It can deadlock if the remote node calls back
```

CLI Commands

```
# All CLI commands — replace 7000 with your port
./chord_bin -mode cli -port 7000 ping
./chord_bin -mode cli -port 7000 put <key> <value>
./chord_bin -mode cli -port 7000 get <key>
./chord_bin -mode cli -port 7000 delete <key>
```

Important Rules

STOP Rules for This Lab

1. Only use YOUR group's containers — they contain your group number. 2. Never run docker stop or docker rm — affects other groups. 3. Never touch k8s_ containers on 10.8.100.2 — system containers. 4. Always pkill chord_bin before recompiling. 5. If the ring is completely broken — tell your instructor, they will run restart_group.sh.

Port Reference

Group	Port	Group	Port	Group	Port	Group	Port
1	7000	6	7005	11	7010	16	7015
2	7001	7	7006	12	7011	17	7016
3	7002	8	7007	13	7012	18	7017
4	7003	9	7008	14	7013	19	7018
5	7004	10	7009	15	7014	20	7019